1주차 2일차: XAML과 기본 레이아웃 — 30분

1. 학습 목표 — 2분

- Grid와 StackPanel로 화면을 배치합니다.
- TextBox, Button, TextBlock의 역할을 구분합니다.
- 입력 영역과 결과 영역이 있는 화면을 만듭니다.

시작 상태는 1일차 완료본입니다.

2. 핵심 개념 설명 — 7분

화면의 작은 구성 요소를 컨트롤이라고 부릅니다. 입력받는 TextBox, 눌러서 실행하는 Button, 글자를 보여 주는 TextBlock부터 사용합니다.

요소	역할	이번 화면에서의 예
TextBox	사용자가 글자를 편집	테스트할 텍스트 입력
Button	사용자의 실행 동작을 받음	실행 버튼
TextBlock	읽을 글자를 표시	안내와 결과

배치는 두 종류만 익힙니다.

Grid는 행과 열로 나누는 배치 영역입니다. 이번에는 창 전체를 안내, 입력, 결과의 세 행으로 나눕니다. 입력 행 안에 작은 Grid를 넣어 입력창과 버튼을 두 열로 나눕니다. Grid는 데이터를 표로 표시하는 DataGrid와 다른 역할입니다.

- Auto: 내용이 필요한 만큼 공간을 사용합니다.
- *: 다른 부분이 사용하고 남은 공간을 사용합니다.
- Grid.Row="1": 두 번째 행에 배치합니다. 행과 열 번호는 0부터 시작하고, 생략하면 0입니다.
- Grid.Column="1": 두 번째 열에 배치합니다.

StackPanel은 자식을 순서대로 쌓는 배치 영역입니다. 기본 방향은 세로입니다. 결과 제목과 결과 문구를 위아래로 쌓는 데 사용합니다.

기본 창은 직접 자식 하나를 받습니다. 그래서 Grid를 하나 넣고 그 안에 여러 요소를 배치합니다. 좌표를 일일이 계산하지 않아도 창 크기에 맞춰 배치가 조정됩니다.

Margin="0,0,8,0"은 왼쪽·위·오른쪽·아래 순서의 바깥 여백입니다. Padding="16,6"은 버튼 내용의 좌우·상하 안쪽 여백입니다. TextWrapping="Wrap"은 긴 글자를 줄바꿈합니다. 꾸미는 방법을 외우기보다 요소가 겹치지 않고 읽히는지만 확인하세요.

3. 짧고 이해하기 쉬운 코드 예제 — 6분

아래는 오늘 완료 시점의 두 파일 전체입니다. 같은 이름의 파일 내용을 각각 교체합니다. 두 파일을 하나로 합치지 않습니다.

MainWindow.xaml

```
<Window x:Class="WpfTestApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="WPF 텍스트 테스트" Height="360" Width="640"
        MinHeight="300" MinWidth="420">
    <Grid Margin="16">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto" />
            <RowDefinition Height="Auto" />
            <RowDefinition Height="*" />
        </Grid.RowDefinitions>

        <TextBlock Text="테스트할 텍스트를 입력하세요."
                   Margin="0,0,0,8" />

        <Grid Grid.Row="1">
            <Grid.ColumnDefinitions>
                <ColumnDefinition Width="*" />
                <ColumnDefinition Width="Auto" />
            </Grid.ColumnDefinitions>

            <TextBox x:Name="InputTextBox"
                     Text="안녕하세요, WPF"
                     VerticalContentAlignment="Center"
                     Margin="0,0,8,0" />
            <Button Grid.Column="1" Content="실행"
                    Padding="16,6" />
        </Grid>

        <StackPanel Grid.Row="2" Margin="0,16,0,0">
            <TextBlock Text="결과" Margin="0,0,0,8" />
            <TextBlock x:Name="ResultTextBlock"
                       Text="아직 실행하지 않았습니다."
                       TextWrapping="Wrap" />
        </StackPanel>
    </Grid>
</Window>
```

MainWindow.xaml.cs

```
using System.Windows;

namespace WpfTestApp;

public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
    }
}
```

오늘 C# 파일은 1일차와 같습니다. x:Name은 다음 시간에 C#에서 해당 요소를 찾을 이름입니다. TextBox와 TextBlock은 Text를, Button은 Content를 화면에 표시합니다. 아직 Button에 Click 연결이 없으므로 버튼을 눌러도 결과는 바뀌지 않습니다.

4. Visual Studio에서 따라 하는 실습 — 12분

1. **2분:** 실행 중인 창을 닫고 MainWindow.xaml을 엽니다. 위 코드에서 세 행과 입력 행의 두 열을 먼저 찾습니다.
2. **5분:** XAML 전체를 교체하고 저장합니다. C# 파일은 그대로 둡니다. 1일차와 다른 날짜의 코드가 섞였다면 위 C# 전체로 맞춥니다.
3. **2분:** F5로 실행합니다. 안내 아래에 입력창과 실행 버튼, 그 아래에 결과 문구가 나타나는지 확인합니다. 입력창의 기본 글자를 지우고 다른 글자를 입력합니다.
4. **2분:** 창을 넓혀 봅니다. 입력창이 넓어지고 버튼은 필요한 너비를 유지하는지 봅니다. 이것이 입력 행의 별표 열과 Auto 열이 하는 일입니다.
5. **1분:** 실행을 종료합니다. TextBox는 편집할 수 있고 결과 TextBlock은 직접 편집할 수 없다는 차이를 확인합니다.

코드가 길어 보여도 대부분은 배치입니다. 이번 시간에는 행 세 개, 입력 행의 열 두 개, 결과의 세로 쌓기만 구분하면 됩니다.

5. 이번 학습에서 반드시 이해해야 할 핵심 내용 요약 — 3분

- Grid는 행과 열로 나누고, StackPanel은 자식을 순서대로 쌓습니다.
- Auto는 필요한 크기, 별표는 남는 공간입니다.
- TextBox는 입력, Button은 실행, TextBlock은 표시를 담당합니다.
- XAML로 버튼을 배치하는 것과 클릭했을 때 할 일을 연결하는 것은 별개입니다.

완료본은 day02 폴더에 있습니다. 다음은 3일차: 입력과 버튼 이벤트입니다.